Chapter 2. n8n 핵심 노드 심화

> ⏱️ **배정 시간 60분** — Trigger 15분 · 분기 노드 10분 · 실습 2-1 15분 · 실습 2-2 10분 · 실습 2-3 10분

📋 학습 목표

이 챕터를 마치면 다음을 할 수 있습니다.

- [ ] Webhook, Schedule, Chat 세 가지 Trigger의 차이와 사용 시점을 설명할 수 있다
- [ ] Publish(발행)가 무엇이고 왜 필요한지 설명할 수 있다
- [ ] Schedule Trigger로 매일 정해진 시각에 실행되는 워크플로우를 만들 수 있다
- [ ] IF 노드로 참/거짓 두 갈래 분기를 만들 수 있다
- [ ] Switch 노드로 세 갈래 이상의 분기를 직접 구성할 수 있다
- [ ] 조건에 따라 다른 내용의 이메일 또는 Slack 메시지를 발송할 수 있다
- [ ] Execution 로그로 오류를 확인하고 원인을 찾을 수 있다

📖 이 장의 용어

용어	영문 · 약어 풀이	설명
Webhook	웹훅	외부에서 특정 URL로 요청을 보내면 그 순간 워크플로우가 실행되는 방식. "전화번호를 알려주고 연락이 오면 받는 것"에 비유
Test URL / Production URL	테스트 주소 / 운영 주소	테스트 주소는 편집 중 1회만 대기, 운영 주소는 Publish 후 항상 동작
Publish / Unpublish	발행 / 발행 취소	편집 중인 초안을 운영 버전으로 확정해 Trigger가 실제로 동작하게 하는 것. Unpublish는 운영 중단
Schedule Trigger	스케줄 트리거, 예약 실행	정해진 시각·주기마다 자동 실행. Timezone 설정을 따름
Timezone	시간대	워크플로우가 "몇 시"를 판단하는 기준. 한국은 `Asia/Seoul`. 미설정 시 미국 시간 기준
Chat Trigger	채팅 트리거	n8n 내장 채팅창에 메시지를 입력하면 실행. 입력값은 `$json.chatInput`

용어	영문 · 약어 풀이	설명
분기	Branch	조건에 따라 흐름을 나눠 다른 처리를 하는 것. IF(2갈래), Switch(3갈래 이상)
연산자	Operator	비교 방식. `is equal to`(같다), `is greater than or equal to`(크거나 같다) 등
타입	Type (String / Number / Boolean / Date)	값의 종류. 문자열 / 숫자 / 참거짓 / 날짜. 숫자 비교를 문자열로 하면 엉뚱한 결과가 나옴
Merge	병합	갈라진 흐름을 다시 하나로 합치는 노드
Credential	자격 증명	외부 서비스에 접속하기 위한 계정·비밀번호·API Key를 n8n에 저장해 둔 것. 한 번 등록해 여러 노드에서 재사용
SMTP	Simple Mail Transfer Protocol	메일을 보낼 때 쓰는 통신 규약. Gmail은 `smtp.gmail.com`, 포트 465
SSL / TLS	Secure Sockets Layer / Transport Layer Security	통신 내용을 암호화하는 기술. "사용함"으로 두면 비밀번호가 암호화되어 전송됨
앱 비밀번호	App Password	Google 계정 비밀번호 대신 외부 프로그램에 주는 16자리 전용 비밀번호. 2단계 인증(2FA)이 켜져 있어야 발급 가능
Incoming Webhook (Slack)	수신 웹훅	Slack이 발급하는 URL로 JSON을 POST하면 채널에 메시지가 올라가는 기능
Body	요청 본문	POST 요청에 담아 보내는 데이터. Slack에는 `{"text": "..."}` 형태의 JSON

2.1 Trigger 종류

Trigger는 워크플로우를 **시작시키는** 노드입니다. Chapter 1에서 쓴 Manual Trigger는 내가 버튼을 눌러야 실행되지만, 실제 업무 자동화는 사람이 없어도 알아서 돌아가야 합니다. 그 역할을 하는 것이 아래 Trigger들입니다.

2.1.1 Webhook Trigger

Webhook은 외부에서 특정 URL로 요청을 보내면 그 순간 워크플로우가 실행되는 방식입니다.

동작 원리

1. Webhook Trigger 노드를 추가하면 n8n이 고유한 URL을 만들어줍니다.
2. 그 URL로 요청이 들어오면 워크플로우가 즉시 실행됩니다.

3. 요청에 담긴 데이터는 다음 노드에서 `$json`으로 바로 사용할 수 있습니다.

URL이 두 개인 이유

구분	언제 동작하나	용도
Test URL	노드의 `Listen for Test Event`를 눌러 대기 중일 때만, 1회	편집 화면에서 개발·테스트
Production URL	워크플로우를 **Publish**한 동안 항상	실제 운영

> 💡 "테스트할 땐 됐는데 배포하니 안 된다"는 문제의 절반이 이 두 URL을 헷갈려서 생깁니다. Chapter 8에서 다시 다룹니다.

간단 확인 실습

1. 새 워크플로우에 `Webhook` 노드 추가
2. HTTP Method: `GET`, Path: `test-webhook`으로 설정
3. `Listen for Test Event` 클릭 (노드가 대기 상태로 바뀝니다)
4. 브라우저 새 탭에서 화면에 표시된 Test URL로 접속

```
http://localhost:5678/webhook-test/test-webhook
```

5. n8n 화면으로 돌아오면 워크플로우가 실행되어 있고, OUTPUT에 요청 정보가 담겨 있습니다

> 지금 만든 Webhook은 내 PC 안에서만 접근할 수 있습니다. 외부 서비스가 내 PC를 직접 호출하게 하려면 별도 설정이 필요하지만, 이 과정에서는 다루지 않습니다. 여기서는 **요청이 오면 실행된다는 원리**만 확인하면 충분합니다.

2.1.2 Schedule Trigger

정해진 시간이나 주기마다 워크플로우를 자동 실행합니다. "매일 오전 8시", "10분마다" 같은 반복 작업에 씁니다. 이 챕터의 실습 2-1에서 본격적으로 사용합니다.

원하는 동작	설정
10분마다 실행	Trigger Interval → `Minutes`, Minutes Between Triggers → `10`
매일 오전 8시	Trigger Interval → `Days`, Days Between Triggers → `1`, Trigger at Hour → `8am`, Trigger at Minute → `0`

원하는 동작	설정
매주 월요일 오전 9시	Trigger Interval → `Weeks`, Trigger on Weekdays → `Monday`, Trigger at Hour → `9am`

> ⚠️ **Schedule Trigger는 워크플로우의 Timezone 설정을 따릅니다.** 설정하지 않으면 미국 시간대 기준으로 돌아 "오전 8시"가 엉뚱한 시각이 됩니다. 실습 2-3에서 Timezone을 `Asia/Seoul`로 바꾸는 절차를 포함했습니다.

Publish란 무엇인가

n8n은 편집 중인 내용을 자동 저장하지만, 저장만으로는 운영에 반영되지 않습니다. **Publish를 눌러야 Trigger가 실제로 동작하기 시작합니다.** Schedule Trigger와 Webhook의 Production URL이 대표적입니다. 편집 화면에서 ▶ 버튼으로 실행하는 것은 "수동 테스트 실행"이며, Publish와 무관하게 언제든 가능합니다.

2.1.3 Chat Trigger

n8n이 제공하는 채팅창을 통해 워크플로우를 실행시키는 Trigger입니다. 사용자가 입력한 메시지가 `$json.chatInput`으로 전달됩니다.

간단 확인 실습

1. 새 워크플로우에 `Chat Trigger` 노드 추가 (검색 시 `On chat message`로 표시될 수 있습니다)
2. 캔버스 하단의 **Open chat** 버튼 클릭
3. 아무 메시지나 입력 후 전송
4. OUTPUT 패널에서 `chatInput` 필드에 입력한 텍스트가 담겼는지 확인

Chapter 4에서 AI Agent와 연결해 실제 챗봇을 만들 때 이 Trigger를 사용합니다.

📊 Trigger 비교표

Trigger	실행 시점	대표 사용 사례
Manual Trigger	▶ 버튼을 눌렀을 때	개발·테스트 중 수동 실행
Webhook Trigger	지정된 URL로 요청이 왔을 때	외부 시스템 연동, 폼 제출
Schedule Trigger	정해진 시간·주기	정기 리포트, 아침 브리핑, 배치 작업
Chat Trigger	채팅창에 메시지를 입력했을 때	AI Agent 챗봇

✅ 체크포인트 질문

- "매일 자정에 어제 데이터를 집계해 이메일로 보낸다"에 맞는 Trigger는? → **Schedule Trigger**
- 외부 웹사이트의 문의 폼이 제출될 때마다 실행하려면? → **Webhook Trigger**
- Schedule Trigger를 설정했는데 아무 일도 일어나지 않는다면 가장 먼저 확인할 것은? → **Publish 여부, 그다음 Timezone**

2.2 조건 분기 노드

같은 워크플로우라도 상황에 따라 다른 처리를 해야 할 때가 많습니다. 그때 쓰는 것이 분기 노드입니다.

2.2.1 IF 노드 — 두 갈래

조건 하나를 두고 참(true)인지 거짓(false)인지에 따라 흐름을 둘로 나눕니다.

설정 방법

1. `IF` 노드 추가
2. `Add Condition` 클릭 → 비교할 값의 타입 선택 (String, Number, Boolean, Date)
3. 좌측(Value 1), 연산자(Operator), 우측(Value 2) 입력

예시: 환율이 1,400원 이상인지

항목	값
Value 1	`{{ $json.usd_krw }}`
Operator	`is greater than or equal to`
Value 2	`1400`

- 조건이 **참**이면 노드의 위쪽 `true` 출력으로 데이터가 흐릅니다
- 조건이 **거짓**이면 아래쪽 `false` 출력으로 흐릅니다

> ⚠️ **타입을 반드시 맞추세요.** 환율은 숫자이므로 조건 타입을 **Number**로 선택해야 합니다. String으로 두면 `"9"`가 `"1400"`보다 크다고 판정되는 등 엉뚱한 결과가 나옵니다. IF 노드가 계속 한쪽으로만 분기한다면 대부분 이 문제입니다.

2.2.2 Switch 노드 — 세 갈래 이상

조건이 셋 이상으로 나뉠 때는 IF를 여러 개 겹치는 대신 **Switch 노드**를 씁니다.

설정 방법

1. `Switch` 노드 추가

2. `Mode`를 `Rules`로 설정
3. `Add Routing Rule`로 분기를 하나씩 추가하고, 각각에 조건을 지정

Switch는 **조건에 맞는 첫 번째 출력으로만** 데이터를 보냅니다. 따라서 구간이 겹치지 않게 순서를 잡아야 합니다.

2.2.3 Merge 노드 — 다시 합치기

나뉜 흐름을 각각 처리한 뒤 하나로 합쳐야 할 때 씁니다.

모드	설명
`Append`	여러 입력의 아이템을 순서대로 이어 붙임
`Combine`	같은 위치(또는 지정한 키)의 아이템끼리 필드를 합침
`Choose Branch`	여러 입력 중 하나만 통과시킴

이 챕터의 실습에서는 나뉜 흐름이 각자 알림을 보내고 끝나므로 Merge를 쓰지 않습니다. **합쳐서 공통 처리가 필요할 때 쓰는 노드**라는 개념만 기억해두세요.

✅ 체크포인트 질문

- 조건이 3개 이상이면 IF와 Switch 중 무엇이 적합할까요? → **Switch 노드**
- IF 노드에서 조건이 거짓일 때 데이터는 어디로 갈까요? → **false 출력(아래쪽)**

2.3 실습 — 매일 아침 8시 날씨·환율 브리핑

> 🎯 **목표**: Chapter 1의 워크플로우를 매일 아침 자동 실행되는 브리핑으로 바꾸고, 환율 구간에 따라 다른 문구를 이메일 또는 Slack으로 발송한다.
> ⏱️ **소요 시간 35분** — 실습 2-1(Schedule + 알림) 15분 → 2-2(IF, 두 갈래) 10분 → 2-3(Switch, 세 갈래 + Publish) 10분

이 실습이 해결하는 업무

구매·해외영업·총무 업무에서는 매일 아침 환율을 확인하고 기준을 넘으면 결제·발주 시점을 조정합니다. 이 확인 작업을 자동화합니다. 날씨는 외근·행사 준비용 부가 정보입니다.

2.3.1 준비 — 알림 채널 정하기

먼저 메시지를 어디로 보낼지 하나를 고릅니다. 둘 다 무료입니다.

> 💡 **어느 쪽을 고를까**: Slack 워크스페이스를 이미 쓰고 계시면 **방법 B**가 빠릅니다(3분). 그렇지 않다면 **방법 A**로 진행하세요. 실습실 네트워크에서 SMTP(465 포트)가 차단되어 있으면 방법 B만 가능합니다.

방법 A. 이메일 (Gmail SMTP)

SMTP(Simple Mail Transfer Protocol)는 메일을 보낼 때 쓰는 통신 규약입니다. n8n이 Gmail 서버에 "이 계정으로 메일을 보내달라"고 요청하는 통로입니다.

Gmail은 일반 비밀번호로 외부 프로그램 로그인을 허용하지 않습니다. **앱 비밀번호**라는 별도의 16자리 비밀번호를 발급받아야 합니다.

1. myaccount.google.com 접속
2. 좌측 **보안** 메뉴에서 **2단계 인증**이 켜져 있는지 확인 — 꺼져 있으면 먼저 켜야 합니다 (앱 비밀번호는 2단계 인증이 전제 조건입니다)
3. 화면 **상단 검색창에 "앱 비밀번호"를 직접 검색** — 보안 페이지 목록에는 잘 보이지 않습니다
4. 앱 이름을 아무렇게나 입력(예: `n8n`) 후 **만들기**
5. 생성된 16자리를 복사해 메모장에 저장

> ⚠️ 이 16자리는 **한 번만 표시**되며 다시 볼 수 없습니다. 창을 닫기 전에 복사하세요. 잃어버리면 재발급하면 됩니다.
> ⚠️ 회사·학교에서 발급받은 Google Workspace 계정이거나 계정에 고급 보호가 켜져 있거나, 2단계 인증을 보안 키로만 설정한 경우에는 앱 비밀번호 메뉴 자체가 나타나지 않습니다. 이 경우 개인 Gmail을 쓰거나 방법 B로 진행하세요.

n8n에서 `Send Email` 노드를 추가하고 Credential(자격 증명: 외부 서비스 접속에 필요한 계정·비밀번호를 n8n에 저장해 두는 것)을 아래와 같이 입력합니다.

항목	값
Host	`smtp.gmail.com`
Port	`465`
User	본인 Gmail 주소
Password	발급받은 앱 비밀번호 16자리 (**공백 없이** 붙여넣기)
SSL/TLS	사용함

방법 B. Slack (Incoming Webhook)

1. Slack Webhook URL 발급 — api.slack.com/apps에서 **Create New App** → **From scratch**로 앱을 만들고 워크스페이스 선택 → 좌측 **Incoming Webhooks**에서 **Activate Incoming Webhooks** 켜기 → **Add New Webhook to Workspace**로 메시지를 받을 채널 선택 → 생성된 Webhook URL 복사. URL을 아는 사람은 누구나 그 채널에 글을 올릴 수 있으므로 비밀번호처럼 다룹니다. 워크스페이스 설정에 따라 앱 설치에 관리자 승인이 필요할 수 있습니다.

2. n8n에서 `HTTP Request` 노드 추가 (Chapter 1에서 쓴 것과 같은 노드입니다)

항목	값
Method	`POST`
URL	발급받은 Slack Webhook URL
Body Content Type	`JSON`
Specify Body	`Using JSON`
JSON	`{{ { "text": $json.briefing } }}`

> 💡 JSON 칸의 `$json.briefing`은 2.3.2에서 만들 필드입니다. 고정 문구가 아니라 **앞 노드의 값**을 넣어야 하므로 Expression 모드(fx)로 작성합니다. 분기마다 다른 문구를 보내려면 이 `briefing` 필드의 내용을 분기별로 다르게 만들면 됩니다.

2.3.2 실습 2-1. Schedule Trigger로 바꾸고 알림 연결하기

완성 흐름

```
Schedule Trigger(매일 8시) → 날씨 조회 → 환율 조회 → 브리핑 작성 → 알림 문구 → 알림 발송
```

1. `실습1_날씨환율조회` 워크플로우를 열고, `···` 메뉴 → **Duplicate**로 복제
2. 복제본의 이름을 `실습2_아침브리핑`으로 변경
3. **Manual Trigger 노드를 삭제**하고, 그 자리에 `Schedule Trigger` 노드 추가 → `날씨 조회` 노드와 연결
4. Schedule Trigger 설정

항목	값
Trigger Interval	`Days`
Days Between Triggers	`1`
Trigger at Hour	`8am`
Trigger at Minute	`0`

5. `브리핑 작성` 노드 뒤에 `Edit Fields (Set)` 노드를 하나 더 추가하고 이름을 `알림 문구`로 변경한 뒤, 아래 두 필드를 추가합니다.

Field Name	Type	Value (Expression 모드)
`briefing`	String	`{{ '[아침 브리핑 ' + $now.setZone('Asia/Seoul').toFormat('MM/dd') +`

Field Name	Type	Value (Expression 모드)
		`']\n' + $json.weather_message + '\n' + $json.rate_message }}`
`usd_krw`	Number	`{{ $json.usd_krw }}`

> ⚠️ **같은 Edit Fields 노드 안에서 만든 필드는 그 노드의 다른 필드에서 참조할 수 없습니다.** `$json`은 노드에 **들어온** 데이터(브리핑 작성 노드의 경우 환율 조회 결과)를 가리키므로, `briefing`을 `브리핑 작성` 노드에 함께 만들면 `undefined`가 들어갑니다. 그래서 노드를 하나 더 둡니다. `usd_krw`를 다시 담는 이유는 Edit Fields가 기본적으로 지정한 필드만 다음 노드로 넘기기 때문입니다. 실습 2-2, 2-3의 조건 분기가 이 값을 사용합니다.

> 💡 `$now`는 현재 시각을 담은 n8n 내장 변수이고, `.setZone('Asia/Seoul').toFormat('MM/dd')`는 한국 시간 기준으로 "10/24" 형태로 바꿉니다. `\n`은 줄바꿈입니다.

6. `알림 문구` 뒤에 2.3.1에서 고른 알림 노드를 연결하고 메시지를 설정

알림 방식	설정
Send Email	To: 본인 주소 / Subject: `{{ '아침 브리핑 ' + $now.setZone('Asia/Seoul').toFormat('MM/dd') }}` / Text: `{{ $json.briefing }}`
Slack(HTTP Request)	JSON: `{{ { "text": $json.briefing } }}`

7. **테스트 실행**: Schedule Trigger 노드 위의 ▶ 버튼을 누르면 예약 시각과 관계없이 **지금 한 번** 실행됩니다. 각 노드에 초록 체크가 뜨고 실제로 메시지가 도착하는지 확인합니다

> ⚠️ Schedule Trigger를 ▶로 실행하는 것은 **테스트**입니다. 매일 8시에 실제로 돌게 하려면 2.3.4의 Publish가 필요합니다.

2.3.3 실습 2-2. IF로 환율 경고 분기 만들기

환율이 1달러 1,400원 이상이면 "해외 결제·발주 보류 검토" 문구를 붙이고, 아니면 평소 브리핑을 보냅니다.

완성 흐름

```
… → 알림 문구 → IF(usd_krw ≥ 1400?) ┬ true  → 알림(경고 문구 포함)
                                     └ false → 알림(평소 브리핑)
```

1. `알림 문구`와 알림 노드 사이의 연결선을 지우고, 그 사이에 `IF` 노드 추가

2. 조건 설정 (타입은 **Number**)

항목	값
Value 1	`{{ $json.usd_krw }}`
Operator	`is greater than or equal to`
Value 2	`1400`

3. 기존 알림 노드는 `false` 출력에 연결
4. 알림 노드를 하나 더 추가해 `true` 출력에 연결하고, 메시지를 다르게 작성

분기	메시지
true (1,400원 이상)	`{{ $json.briefing + '\n\n[환율 주의] 1,400원을 넘었습니다. 해외 결제·발주 시점을 검토하세요.' }}`
false (1,400원 미만)	`{{ $json.briefing }}`

> 💡 IF 노드는 데이터를 바꾸지 않고 통과시키므로, IF 뒤에서도 `$json.briefing`과 `$json.usd_krw`를 그대로 쓸 수 있습니다.

5. Schedule Trigger의 ▶ 버튼으로 전체 실행 → 어느 분기로 흘렀는지 Canvas에서 확인하고, 실제로 메시지가 도착했는지 확인

> 💡 **두 분기를 모두 테스트하려면** IF의 기준값(Value 2)을 잠깐 `0`이나 `9999`로 바꿔 반대쪽 분기를 강제로 태워봅니다. 확인 후 `1400`으로 되돌립니다.

2.3.4 실습 2-3. Switch로 세 구간 분기 + Publish

두 갈래로는 "주의"와 "평소"밖에 구분하지 못합니다. **유리 / 보통 / 주의** 세 구간으로 나눠봅니다. IF를 두 개 겹칠 수도 있지만, 구간이 늘어날수록 지저분해집니다. Switch가 이럴 때 쓰는 노드입니다.

1. IF 노드를 삭제하고 그 자리에 `Switch` 노드 추가
2. `Mode`를 `Rules`로 설정
3. `Add Routing Rule`을 세 번 눌러 아래 조건을 차례로 입력 (모두 **Number** 타입)

순서	조건	Rename Output
1	`{{ $json.usd_krw }}` `is less than` `1300`	`유리`
2	`{{ $json.usd_krw }}` `is less than` `1400`	`보통`

순서	조건	Rename Output
3	`{{ $json.usd_krw }}` `is greater than or equal to` `1400`	`주의`

> 💡 **두 번째 조건이 왜 `< 1400` 하나로 충분한가**: Switch는 위에서부터 검사해 **처음 맞는 곳에서 멈춥니다.** 1,300원 미만은 이미 1번에서 걸러졌으므로, 2번에 도달했다는 것은 곧 1,300원 이상이라는 뜻입니다. `1300 이상 그리고 1400 미만`을 따로 쓸 필요가 없습니다. 이것이 Switch의 조건을 짤 때 가장 중요한 감각입니다.

4. `Rename Output`으로 각 출력에 이름을 붙이면 캔버스에서 어느 분기인지 한눈에 보입니다

5. 세 출력에 각각 알림 노드를 연결하고 메시지를 다르게 작성

분기	메시지
유리 (1,300원 미만)	`{{ $json.briefing + '\n\n[환율 유리] 해외 결제·선구매에 유리한 구간입니다.' }}`
보통 (1,300~1,400원)	`{{ $json.briefing }}`
주의 (1,400원 이상)	`{{ $json.briefing + '\n\n[환율 주의] 1,400원을 넘었습니다. 해외 결제·발주 시점을 검토하세요.' }}`

6. 기준값을 바꿔가며 세 분기가 모두 동작하는지 확인하고 원래 값으로 되돌립니다

Timezone 설정과 Publish

7. 우측 상단 `···` → **Settings** → **Timezone**을 `Asia/Seoul`로 변경하고 저장. 이 설정이 없으면 "8am"이 미국 시간 기준으로 실행됩니다

8. 우측 상단 **Publish** 클릭 → 버전 이름 입력 팝업이 뜨면 그대로 확인

9. 버튼 상태가 **Published**로 바뀐 것을 확인합니다. 이 상태에서 n8n이 켜져 있으면 내일 아침 8시에 자동으로 브리핑이 발송됩니다

> ⚠️ **실습실 PC는 수업 후 종료되므로**, 확인이 끝나면 Publish 버튼 옆 드롭다운(▾) → **Unpublish**로 되돌립니다. 집이나 회사 PC에서는 Publish 상태로 두고 n8n을 켜두면 매일 아침 브리핑을 받을 수 있습니다. Chapter 6의 오류 처리 실습에서 이 워크플로우를 다시 Publish합니다.

2.3.5 테스트 및 디버깅

Execution 로그 활용

확인 항목	방법
실행 성공/실패	Executions 목록의 상태 아이콘 (초록/빨강)
어느 노드에서 오류가 났는지	실행 항목 클릭 → 빨간색으로 표시된 노드
각 노드의 입출력 데이터	노드 클릭 → INPUT/OUTPUT 패널

자주 발생하는 오류

증상	원인	해결
IF/Switch가 항상 한쪽으로만 분기	조건 타입이 String으로 되어 있음	조건 타입을 **Number**로 변경
Value 1이 비어 있다고 나옴	Expression 경로 오타, 또는 `usd_krw` 필드를 Number가 아닌 String으로 만듦	INPUT 패널에서 실제 필드명·타입 확인
알림 본문에 `undefined`가 표시됨	`briefing`을 `브리핑 작성` 노드 안에서 만듦	`알림 문구` 노드를 따로 두고 그곳에서 `briefing` 작성 (2.3.2 5번)
`weather_message`가 비어 있음	`$('날씨 조회')`의 노드 이름이 실제 이름과 다름	캔버스의 노드 이름을 확인해 Expression 수정
이메일이 발송되지 않음	앱 비밀번호 오입력 (공백 포함 등), 또는 465 포트 차단	16자리를 공백 없이 다시 붙여넣기 / Slack으로 전환
Slack 메시지가 오지 않음	Body 형식 오류, JSON 칸이 Expression 모드가 아님	Body Content Type `JSON` 확인, fx 모드 확인
8시에 실행되지 않음	Publish 안 함, 또는 Timezone 미설정	Publish 상태와 Settings의 Timezone 확인

🎉 완성 확인

Schedule Trigger의 ▶ 실행 시 환율 구간에 따라 세 가지 중 하나의 문구가 이메일 또는 Slack으로 도착하고, Settings의 Timezone이 `Asia/Seoul`이면 실습 완료입니다.

📌 Chapter 2 요약

개념	핵심 내용
Webhook Trigger	외부 요청이 오면 실행. Test URL과 Production URL 구분
Schedule Trigger	정해진 시간·주기 자동 실행. **Publish + Timezone** 둘 다 필요
Chat Trigger	채팅 입력으로 실행. Chapter 4 Agent의 기반

개념	핵심 내용
Publish	저장은 초안일 뿐, Publish해야 Trigger가 실제로 동작
IF 노드	조건 하나로 두 갈래 분기. **조건 타입(Number/String) 주의**
Switch 노드	세 갈래 이상 분기. 위에서부터 검사해 첫 번째 일치에서 멈춤
Merge 노드	나뉜 흐름을 다시 합칠 때 사용
`$now`	현재 시각. `.setZone().toFormat()`으로 한국 시간 표기
실습 2-1~2-3	Schedule → 날씨·환율 조회 → 브리핑 작성 → 알림 문구 → Switch(3구간) → 구간별 알림 발송

다음 Chapter 예고: Chapter 3에서는 Google Gemini API를 n8n에 연결하고, 회의록을 AI에게 넘겨 요약·액션 아이템 추출·공지 메일 초안까지 만드는 워크플로우를 만듭니다. 지금까지는 정해진 규칙대로만 움직였다면, 이제부터 "이해"가 들어갑니다.